```
    return;
  }
  xhr.open(method, url);

  xhr.onerror = function() {
    alert('There was an error.');
  };

  xhr.onload = function() {
    var data = JSON.parse(xhr.responseText);
    if (data.stat == 'ok') {
      var photosDiv = document.getElementById('photos');
      photosDiv.innerHTML = '';
      var photos = data.photos.photo;
      for (var i = 0; i < photos.length; i++) {
        var img = document.createElement('img');
        img.src = photos[i].url_q;
        photosDiv.appendChild(img);
      }
    } else {
      alert(data.message);
    }
  };

  xhr.send();
}
</script>
</body>
</html>
```

Assigns a method to onerror event handler

If you're curious to understand why a cross-origin request failed, some browsers (such as Chrome) will display the reason in the console log. This message will give more context to a human who is able to read the message, but there is no way to read this message from JavaScript. Chapter 7 will go into more detail on how to debug and fix a failing CORS request.

OTHER EVENT HANDLERS

Together, the `onload` and `onerror` event handlers will handle most of your needs. But the `XMLHttpRequest` object exposes a few more event handlers that you can hook into, as shown in table 2.4.

Table 2.4 `XMLHttpRequest` event handlers

Event handler	Function
`onloadstart`	Fires when the request is successfully initiated asynchronously.
`onloadend`	Similar to `onloadstart`, `onloadend` always fires when the request ends. This event handler is useful if you want to execute some piece of code at the very end of the request, regardless of whether or not the request is successful.
`onabort`	Fires if the client aborts the request by calling the `abort` method. If `onabort` fires, the `onload` and `onerror` event handlers will not fire.